Lesson 14: Raising the Stakes

Godot Engine

VIDEO GAME CREATION

This week: later waves get genuinely harder - more enemies, moving faster - using the exact wave variable you've had since Lesson 11.

Open your own [YourName]Game project from Lesson 13 - don't start a new one.

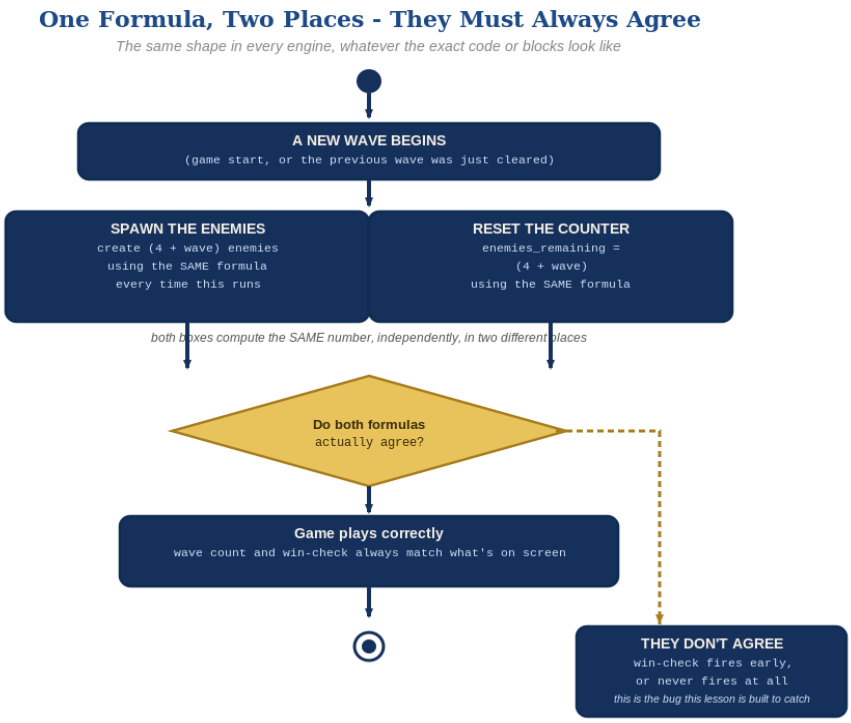
Tip: no new assets this week - difficulty scaling reuses your exact existing enemy sprites and spawn logic.

Setup

1. Open your [YourName]Game project from Lesson 13.
2. No new variables needed for Level 1 - Level 2 will use enemy.tscn's existing speed variable (from Lesson 7).

Level 1 - Minimal (MVP)

Each wave spawns one more enemy than the last (5, 6, 7). Speed stays the same for now - that's Level 2.



Whether it's two places (most engines) or the loop and the reset living in genuinely different scripts (Godot, GameMaker), the rule is the same: one formula, referenced everywhere.

VIDEO GAME CREATION

Whether the spawn loop and the reset live in two places or two genuinely different scripts, the rule is the same: one formula, referenced everywhere.

3. In Main's spawn_enemies(), change the fixed "for enemy_count in range(5)" loop into "for enemy_count in range(4 + $GameManager.wave)" (code below) - wave 1 spawns 5, wave 2 spawns 6, wave 3 spawns 7. spawn_enemies() lives on Main, not GameManager, so wave needs the $GameManager prefix here, the same as every other cross-node reference in this function.
4. Wherever enemies_remaining gets reset to 5 for a new wave - inside GameManager's start_game() and inside on_enemy_destroyed()'s wave-advance else branch - change that to "enemies_remaining = 4 + wave" too (no prefix needed there, since those run directly on GameManager's own script) - all three places must always agree. In start_game() specifically, double-check that "wave = 1" already runs BEFORE this line (it may currently come after) - otherwise enemies_remaining would compute from whatever wave was left over from the previous game, not a fresh wave 1.
5. Run the scene and play through a full round - wave 2 should visibly spawn one more enemy than wave 1, and wave 3 one more than that.

Highlighted lines are new this week; everything else is code you already have.

```
# Main scene's script - spawn_enemies() (revised):
func spawn_enemies():
    for enemy_count in range(4 + $GameManager.wave):
        var new_enemy := enemy_scene.instantiate()
        new_enemy.position = Vector2(150 + enemy_count * 200, 50)
        add_child(new_enemy)
        new_enemy.destroyed.connect($GameManager.on_enemy_destroyed)
```

```
        new_enemy.escaped.connect($GameManager.on_enemy_escaped)

# GameManager's script - start_game() (revised, enemies_remaining line
only):
enemies_remaining = 4 + wave

# GameManager's script - on_enemy_destroyed()'s wave-advance else branch
(revised, enemies_remaining line only):
enemies_remaining = 4 + wave
```

Level 2 - Complete (Alpha/Beta)

Enemy speed ALSO increases each wave, on top of the growing count. Restarting correctly returns everything to wave 1's original values.

- In spawn_enemies(), right after instantiating each enemy, set its speed based on the wave: new_enemy.speed = 60.0 + ($GameManager.wave - 1) * 20.0.
- Confirm enemy.tscn's own speed variable (from Lesson 7) is still what its _process(delta) movement actually reads - no changes needed there.
- Restart mid-game and confirm both count and speed return correctly to wave 1's values.
- Play a full round and confirm wave 3 is noticeably busier and faster than wave 1, but still fair.

Level 3 - Refined - Choose ONE Stretch Feature

Pick exactly one of the five ideas below. These are intentionally open-ended - a good-faith attempt at the general idea is a complete Level 3 finish, not a fully polished feature.

- Destructible shields/barriers: a few simple blocks placed between the player and the enemies that disappear (or shrink) after absorbing a hit or two - reuses your existing collision-detection and hit-reaction patterns from Lessons 4 and 13, just on a new kind of object.
- A power-up: an item that occasionally drops or spawns, which the player collects to temporarily change how shooting works (faster fire rate, multiple bullets at once, or a brief shield) - reuses your existing timer/cooldown pattern from Lesson 3.
- A bonus enemy: a rare, faster-moving, or oddly-patterned enemy worth extra points, spawned occasionally alongside a normal wave - reuses this week's own speed/count scaling ideas, just for one special case instead of the whole wave.
- Formation-marching movement: instead of each enemy independently drifting down (this course's approach since Lesson 4), have the whole wave move together as a group and occasionally step sideways - a genuinely different movement pattern, inspired by classic arcade shooters.
- A title screen and/or game-over screen: a proper opening screen (press a key to start) and/or a dedicated game-over screen with a final score, instead of just text overlaid on the gameplay - reuses your existing game_active flag and UI tools from Lessons 9-10. Godot's Tween node is a natural fit for a smooth title/game-over screen transition.

If Something's Wrong

- Every wave still spawns exactly 5: confirm the loop actually uses "range(4 + $GameManager.wave)" and not a leftover range(5).

- WinText fires early or never: confirm enemies_remaining is reset using the SAME formula everywhere it's set, not 5 in one spot and 4 + wave in another - and confirm wave = 1 runs before enemies_remaining's reset inside start_game().
- Speed doesn't change between waves: confirm enemy.speed is being set INSIDE spawn_enemies() on each new instance, not left at enemy.tscn's default forever.

Want More?

Student Resource List > Godot section, or docs.godotengine.org